

EE 364B Final Report: Convex Framework For Data Subset Selection for LLM Fine-Tuning

Marc Moussa Nasser

1 Introduction

Large language models (LLMs) achieve state-of-the-art performance across a wide range of tasks, largely due to their ability to effectively understand and generate natural language. However, fine-tuning these models often needs datasets containing tens or hundreds of thousands of samples, requiring substantial compute and memory, making it inaccessible to many.

To address this, several lines of work have explored selecting a high-quality subset of training data that aims to retain task performance while significantly reducing training cost (memory, time, computation resources) [1]. Some methods try to match statistical properties of the full data distribution, including mean [2] and covariance [3]. Separately, gradient-based subset selection approaches such as GradMatch [4] aims to approximate the gradient of the full validation (or training) set to identify the most informative examples, achieve cheaper training and faster convergence.

In this work, we inspire from prior these ideas and reformulate them into a single convex optimization framework for subset selection in the fine-tuning regime. Our formulation differs in how these objectives are defined, combined, and formulated, as well as having additional constraints. Given the scale of modern datasets, classical solvers like CVXPY are impractical. Instead, we derive our method using projected proximal gradient descent algorithm such as FISTA [5] with a projection step to efficiently solve the constrained optimization problem. Our experiments focus on BERT-based classification.

2 Problem Formulation

2.1 Constructing the Problem

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$ be the training set, and let $\mathcal{D}_{\text{val}} = \{(x_j^{\text{val}}, y_j^{\text{val}})\}_{j=1}^m$ be a held-out validation set. We assume that the validation set has been carefully chosen to get a desired sample distribution and be representative of the dataset [6]

Our goal is to select a weighted subset of training examples, represented by a weight vector $w \in \Delta^{n-1} := \{w \in \mathbb{R}^n \mid w_i \geq 0, \sum_{i=1}^n w_i = 1\}$, the standard probability simplex in $\mathbb{R}^n$ (n is the number of samples), such that we minimize performance loss on the validation set while using only a subset of the training data for fine-tuning.

We define an objective that aims to match the validation distribution using approximate mean and covariance alignment, and to maximize a gradient alignment score. Our alignment score is novel compared to GRADMATCH [4], which minimizes the ℓ_2 distance between gradients, whereas

our approach emphasizes directional alignment (using dot product). Our formulation and justification is detailed in a few paragraphs below.

2.2 Defining Parameters and Variables

We define the following parameters:

- $X \in \mathbb{R}^{n \times d}$ is the sample-feature(embedding) matrix for the training set, where each row $x_i \in \mathbb{R}^d$ is the embedding of sample i .
- $\mu \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d \times d}$ are the empirical mean and covariance of the validation features.
- $s \in \mathbb{R}^n$ is the gradient alignment score for each training point, defined as $s_i = \frac{1}{T} \sum_{t=1}^T \tilde{\mathbf{g}}_{\text{val}}^{(t)\top} \tilde{\mathbf{g}}_i$, where $\tilde{\mathbf{g}}_{\text{val}}^{(t)\top}$ is validation batch t 's gradient and $\tilde{\mathbf{g}}_i$ is the gradient at sample i (both normalized). Justification of our s formula is found below.
- $p_c \in [0, 1]$ is the target proportion for a category c of validation labels, and δ is a tolerance margin. The constraint ensures a desired repartition of labels in our subset. The category depends on the fine-tuning objective. For example, in classification, one category maps to one class.

We define the following optimization variable

- $w \in \mathbb{R}^n$ is the weight vector over the training samples.

2.3 Defining Problem

Our optimization problem is:

$$\begin{aligned}
\min_{w \in \mathbb{R}^n} \quad & \underbrace{\|X^\top w - \mu\|_2^2}_{\text{mean matching}} + \beta \underbrace{\left\| (X - \mathbf{1}\mu^\top)^\top \text{diag}(w)(X - \mathbf{1}\mu^\top) - \Sigma \right\|_F^2}_{\text{covariance matching}} - \alpha \underbrace{s^\top w}_{\text{gradient score}} + \tau \underbrace{\|w\|_1}_{\text{regularization}} \\
\text{s.t.} \quad & \sum_{i=1}^n w_i = 1, \\
& 0 \leq w_i \leq 1 \quad \text{for all } i, \\
& p_c - \delta \leq \mathbf{1}_c^\top w \leq p_c + \delta \quad \text{for all } c
\end{aligned}$$

where $\mathbf{1}_c$ is the indicator function of category c and $\beta, \alpha, \tau, \delta$ are the hyper-parameters of the multi-objective optimization problem.

2.4 Modeling Choices

Choosing samples: Once we solve this problem, we can select our subset using various methods, such as picking the samples corresponding to the top K weights (since our objective assigns higher weights to samples of bigger importance) or sampling.

Defining s : intuition and justification. The score vector $s \in \mathbb{R}^n$ quantifies how well each training example aligns with the validation objective.

Assuming one gradient step per epoch, full validation loss $\mathcal{L}_{val}$ computation and training sample loss ℓ , define for each training example (x_i, y_i) the one-step alignment:

$$s_i = \nabla_{\theta} \mathcal{L}_{val}(\theta)^{\top} \nabla_{\theta} \ell(f_{\theta}(x_i), y_i),$$

where θ are the current model parameters. A Taylor expansion shows that minimizing validation loss leads to minimizing $-s^{\top} w$, with $g(w) = \sum_i w_i \nabla_{\theta} \ell(f_{\theta}(x_i), y_i)$ (Taylor: $\mathcal{L}_{val}(\theta - \eta g(w)) \approx \mathcal{L}_{val}(\theta) - \eta \nabla_{\theta} \mathcal{L}_{val}(\theta)^{\top} g(w)$).

In practice, since we update over T batches per epoch, we average validation gradients:

$$s_i = \tilde{g}_{val}^{\top} \nabla_{\theta} \ell(f_{\theta}(x_i), y_i), \quad \tilde{g}_{val} = \frac{1}{T} \sum_{t=1}^T \nabla_{\theta} \mathcal{L}_{val}^{(t)}(\theta).$$

Convexity of our problem. Each term in the objective is convex in w . The mean and covariance matching terms are norms of affine functions and therefore convex. The regularization term is convex as it is the L1 norm of our variable. The gradient score is affine. The difference between a the sum of convex functions and an affine function is convex, making our objective convex. The inequality and equality constraints are all affine functions and therefore convex. Therefore our problem is a convex problem.

Iterative runs. We aim to solve this optimization problem efficiently enough to run it periodically during training to take into account the change in the LLM model weights which impacts the gradient scores. If forward/backward passes are too expensive, we can either run the optimization only once/a few selected times, freeze the model backbone and focus on fine-tuning layers, or explore alternative proxy gradient methods.

3 Solving the Problem for Large Datasets: Projected FISTA

Our X matrix is of size $n \times d$. For LLMs, d often ranges from 100 to 800, and $n \geq 10000$ in most cases. This causes convex solvers to be very slow and impractical often crashing. To scale our approach, we choose to use projected proximal gradient method FISTA [5] which is very efficient when a non-smooth term (L1-reg) is present in the objective, beating other methods such as mirror descent.

3.1 Methodology

FISTA for composite optimization. Our objective function has the composite structure:

$$\min_{w \in \mathcal{C}} f_{\text{smooth}}(w) + \tau \|w\|_1,$$

where f_{smooth} includes the mean match, covariance match, and gradient alignment terms, and $\|w\|_1$ promotes sparsity. The constraint set $\mathcal{C}$ is convex, consisting of the simplex constraint $\sum_i w_i = 1$, elementwise bounds $0 \leq w_i \leq 1$, and class-balance bounds $\sum_{i: y_i=c} w_i \in [p_c - \delta, p_c + \delta]$.

Projected FISTA iteratively applies the following updates:

1. Proximal Gradient step: $\tilde{w}^{(k+1)} = \text{Prox}_{\eta\tau\|\cdot\|_1}(z^{(k)} - \eta \nabla f_{\text{smooth}}(z^{(k)}))$

2. Projection onto $\mathcal{C}$: $w^{(k+1)} = \Pi_{\mathcal{C}}(\tilde{w}^{(k+1)})$
3. Update q : $q^{(k+1)} = \frac{1 + \sqrt{1 + 4(q^{(k)})^2}}{2}$
4. Momentum update: $z^{(k+1)} = w^{(k+1)} + \frac{q^{(k)} - 1}{q^{(k+1)}}(w^{(k+1)} - w^{(k)})$

Gradient computation. Let $X_c = X - \mathbf{1}\mu^\top$. Then the gradient is

$$\nabla f_{\text{smooth}}(w) = 2X(X^\top w - \mu) + 2\beta \text{diag}(X_c (X_c^\top \text{diag}(w)X_c - \Sigma) X_c^\top) - \alpha s.$$

(derivation in Appendix)

Proximal step. The proximal operator of L1 norm is the soft-thresholding operator:

$$\text{Prox}_{\eta\tau\|\cdot\|_1}(u) = \text{sign}(u) \cdot \max(|u| - \eta\tau, 0)$$

Projection step. We wish to compute

$$\text{Proj}_C(v) = \arg \min_{w \in [0,1]^n} \frac{1}{2} \|w - v\|_2^2 \quad \text{s.t.} \quad \sum_{i=1}^n w_i = 1, \quad \sum_{i:y_i=c} w_i \in [p_c - \delta, p_c + \delta] \quad \forall c.$$

Let $C_\Delta = \{w | \sum_{i=1}^n w_i = 1, w_i \geq 0 \forall i\}$, $C_{\text{class}} = \{w | \sum_{i:y_i=c} w_i \in [p_c - \delta, p_c + \delta] \forall c\}$. Then $C = C_\Delta \cap C_{\text{class}}$. As we are dealing with convex sets, we can use Dykstra's alternating projection algorithm [7]. To project onto the probability simplex we use Condat's algorithm [8].

To project onto the class-sum constraint set we apply, for each class c :

$$\text{Proj}_{C_{\text{class}}}(w)_i = w_i + \frac{\tau_c - \sum_{j:y_j=c} w_j}{n_c}, \quad \text{where } \tau_c = \text{clip}\left(\sum_{j:y_j=c} w_j, p_c - \delta, p_c + \delta\right),$$
 and n_c is the number of examples with label c (shift every element in w by the same amount to make w feasible). A significant advantage of Dykstra is that it does not require strict feasibility (which is the case in our problem), and as seen in Figure 1, residuals are on the order of 10^{-2} after less than 100 iterations, which is all we need for our problem.

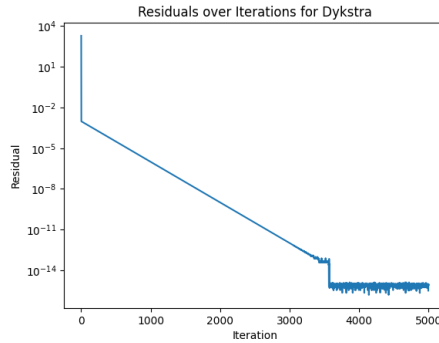


Figure 1: Dykstra Convergence For Our Constraints

Step Size. We determine the step size using backtracking line search, where we decrease η until the following condition is satisfied: $f(w_{k+1}) \leq f(w_k) + \langle \nabla f(w_k), w_{k+1} - w_k \rangle + \frac{1}{2\eta_k} \|w_{k+1} - w_k\|^2$.

3.2 Solver Performance

Convergence and Complexity. FISTA iterates satisfy

$$F(w^{(k)}) - F(w^*) = O(1/k^2) \implies k = O(1/\sqrt{\varepsilon}).$$

Each outer iteration costs

$$\underbrace{O(n d^2)}_{\text{gradient}} + \underbrace{O(n \log(1/\varepsilon_p))}_{\text{projection (Dykstra): simplex + class-sum}} \approx O(n d^2),$$

where ε_p is the inner-projection tolerance. A more detailed derivation is in the Appendix.

comparison with cvxpy We obtain the results seen in table 1:

Table 1: Execution Time and Objective Value for Projected PGD vs. CVXPY

n	Execution Time (s)		Objective Value	
	PGD	CVXPY	PGD	CVXPY
100	0.13	3.2	51.6	51.55
200	0.13	11	25.14	25.09
300	0.15	21	16.6	16.54
400	0.18	36	12.15	12.10

As you can see, our implementation is order of magnitudes faster compared to CVXPY, while having negligible optimum difference when using stopping criterion of $f_k - f_{k-1} \leq \epsilon = 10^{-4}$. Since we do not need extreme accuracy for our problem, we can significantly speed up our algorithm by having a looser stopping criterion.

The runtime for a full dataset (67348 samples, SST dataset [9]) is 19.5 seconds with $\epsilon = 10^{-3}$. Meanwhile, CVXPY crashed. An example convergence plot of our projected FISTA is shown in Figure 2.

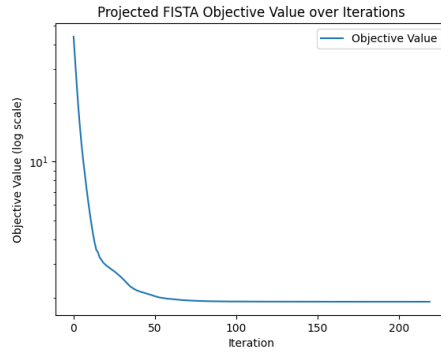


Figure 2: Projected FISTA convergence plot

4 Results & Analysis

4.1 Experiments Environment

Classification with BERT-tiny. In our experiments, x_i is the [CLS] embedding from pretrained BERT-tiny model for training sample i , and d is the embedding size. Due to the volume of experimentation required by the novelty of our approach (to polish the objective, alignment method, and hyperparameters) I used BERT-tiny.

Datasets. We use three datasets: SST [9], which has 2 classes; AG News and Yelp Reviews [10], which have 4 and 5 classes respectively.

Weights Selection Similar to Table 1, I decided to run the classification fine-tuning of BERT Tiny for 3 epochs using the optimal weights obtained from our objective **without** gradient matching yet. I used the top K strategy with $K = [1, 10, 15, 25, 50]\%$ of the dataset size, and compared its performance against random selection. We get 0.36% of w^* to be non-zero within tolerance. We observe the following results:

4.2 Mean and Covariance Matching Results

SST Dataset Results (2 classes)

Subset Size (%)	FISTA weights				Random			
	Time (s)	Train Acc	Eval Acc	Diff	Time (s)	Train Acc	Eval Acc	Diff
1	1.5	0.59	0.56	0.028	1.4	0.58	0.51	0.075
10	9.1	0.82	0.78	0.041	8.2	0.82	0.75	0.071
15	15	0.85	0.80	0.053	12	0.85	0.77	0.076
25	21	0.87	0.81	0.057	21	0.87	0.78	0.090
50	41	0.90	0.82	0.076	40	0.90	0.81	0.090

Table 2: Comparison of FISTA vs Random Subset Selection Strategies (SST Dataset)

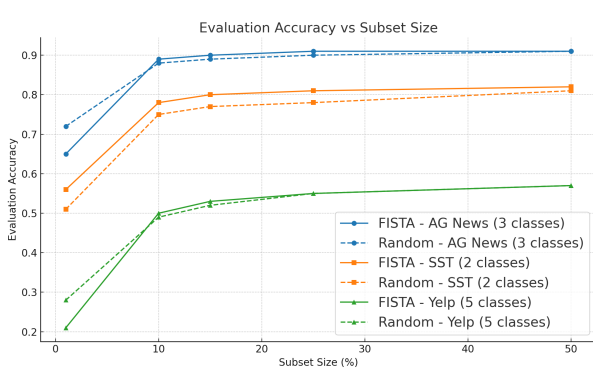
For SST2, we can see that FISTA consistently outperforms Random in validation accuracy across all subset sizes. Smaller FISTA subsets beat larger Random ones ($\text{AccPGD}_{10} > \text{AccRAND}_{15}$, etc.), showing the effectiveness of our selection. Training accuracy remains similar across methods, as expected. Accuracy gains taper at 50%, as most informative points were already selected.

All Datasets Results. We test the same selection strategy for AG News and Yelp datasets and observe the following results in Figure 3a.

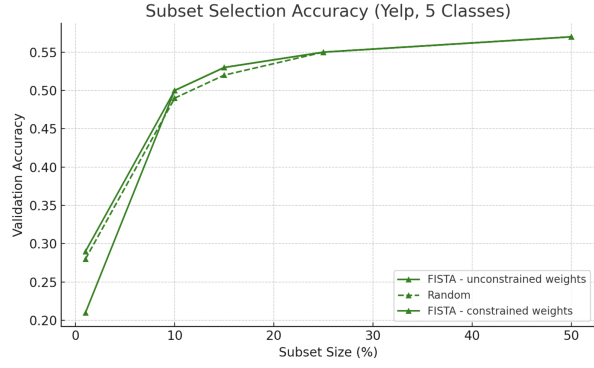
We observe consistent validation improvements at the 10–15% subset level, balancing performance and efficiency. As subset size increases, random selection becomes competitive, since with $n = 60000$, a 50% subset is already fairly representative.

At the 1% level, however, performance drops in multi-class settings due to class imbalance among the top-weighted samples. While w satisfies constraints globally, class-wise balance is not enforced. To fix this, we adjust the strategy to select top- k weights that also meet per-class bounds.

With this change, as shown in Figure 3b, results at $k=1$ improve, now outperforming random, and larger subsets perform equally as the unconstrained method, indicating class balance was already satisfied at those levels.



(a) Mean and Covariance Matching Results



(b) Updated selection strategy on YELP

Figure 3: Comparison of matching objectives and subset selection strategies.

Potential Improvements. There are two key reasons why performance gains are limited for multi-class settings and large subset sizes. (1) With $n = 60000$, random subsets larger than 25% already approximate the data well. In larger datasets ($n \gg 10^5$), we expect much more complex / wide distribution (larger datasets are often associated with more complicated tasks in finetuning), and a random 25-50% of the data would be less representative of the distribution, leading to our subset selection to have better gains. (2) Mean and covariance matching may be insufficient in 128 dimensions. As shown in Figure 4, Yelp’s weight distribution is near-Gaussian with low variance, unlike SST where weights vary meaningfully and eliminated many more samples. This suggests the method doesn’t as strongly identify informative samples in Yelp, mainly for larger k (in our case, 15%). Dimensionality reduction (e.g., PCA) or other distribution matching method could improve this, if we do not see expected gains from (1).

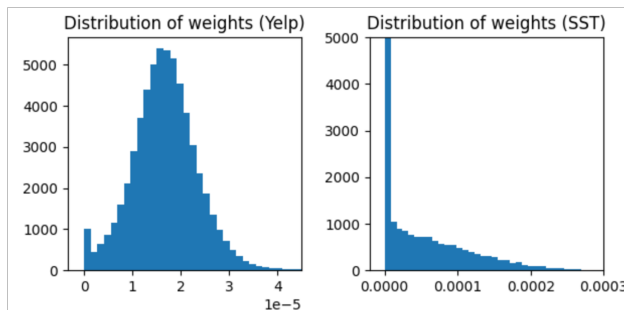


Figure 4: Distribution of Yelp weights and SST weights

4.3 Gradient Matching Results

As previously stated, the main purpose of gradient matching is to boost validation accuracy **per epoch**. The goal is to end up with faster convergence of validation set, requiring less training epochs. We observe Figure 5, showcasing the validation scores after one training epoch. Gradient matching consistently improves accuracy or after one epoch, or matches it (when top- K don’t change after adding GM). This makes it a promising strategy for faster training. To reduce the cost of computing s on large models, we can compute it selectively (not every epoch) or freeze

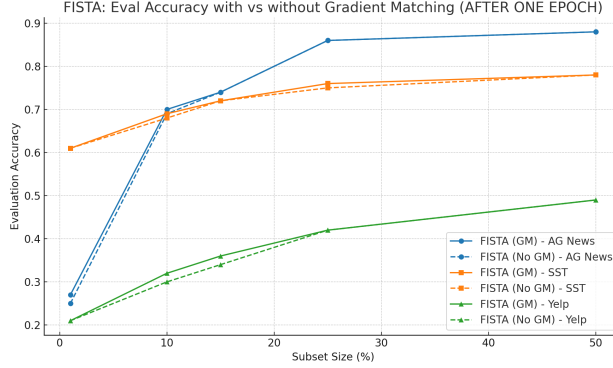


Figure 5: Validation Scores with and without Gradient Alignment

the backbone using only gradients from classification layers (which is what we did). Other proxy gradient methods are also worth exploring.

5 Conclusion & Future Work

We introduced a novel convex optimization framework for subset selection in LLM fine-tuning, combining mean and covariance matching with gradient alignment to improve efficiency and generalization.

To solve the resulting objective with a non-smooth term, we use projected FISTA, with the projection step implemented via Dykstra’s algorithm, which proved fast and reliable under our constraints.

Using a top- K weight selection strategy, we observe consistent gains at the 10–15% subset level, with diminishing returns beyond that, as random selection becomes competitive at this scale ($n = 60,000$). We expect greater improvements on larger datasets, as they are often correlated with a more complex / wide distribution in finetuning, reducing the effectiveness of random sampling. At the 1% level, performance drops in multi-class settings due to class imbalance, which we address by enforcing per-class constraints, improving low- k results. Gains may remain limited when weights follow a narrow, gaussian low-variance distribution, as observed in Yelp.

Future work will explore scaling to larger datasets, alternative distribution matching strategies, improved subset selection heuristics once weights are computed, and cheap proxy model gradient methods.

References

- [1] Mingzhang Xia, Shyamgopal Karthik Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. Less: Selecting influential data for targeted instruction tuning. *arXiv preprint*, arXiv:2402.04333, 2024.
- [2] Alaa Maalouf, Ibrahim Jubran, and Dan Feldman. Introduction to coresets: Approximated mean. *arXiv preprint arXiv:2111.03046*, 2021.
- [3] Olivier Bachem, Mario Lucic, and Andreas Krause. Scalable and provable estimation of joint covariance matrices for high-dimensional data. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2018.
- [4] Krishna Killamsetty, Dheeraj Sivasubramanian, Manikandan Sivakumar, Irina Rish, Moin Agarwal, Ravi Varma, Suvrit Sra, and Srinivasan Parthasarathy. Grad-match: Gradient matching based data subset selection for efficient learning. In *International Conference on Machine Learning (ICML)*, pages 5464–5474, 2021.
- [5] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [6] Ravi Raju, Swayambhoo Jain, Bo Li, Jonathan Li, and Urmish Thakker. Constructing domain-specific evaluation sets for llm-as-a-judge. *arXiv preprint arXiv:2408.08808*, 2024.
- [7] Richard L. Dykstra. An algorithm for restricted least squares regression. *Journal of the American Statistical Association*, 78(384):837–842, 1983.
- [8] Laurent Condat. Fast projection onto the simplex and the ℓ_1 ball. *Mathematical Programming*, 158(1-2):575–585, 2016.
- [9] Richard Socher, Alex Perelygin, Jean Y Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642. Association for Computational Linguistics, 2013.
- [10] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, volume 28, pages 649–657, 2015.

Appendix

A Gradient Derivation

We start from the smooth part of the objective (omitting the ℓ_1 term):

$$f_{\text{smooth}}(w) = \underbrace{\|X^\top w - \mu\|_2^2}_{f_1(w)} + \underbrace{\beta \|X_c^\top \text{diag}(w) X_c - \Sigma\|_F^2}_{f_2(w)} - \alpha s^\top w,$$

where

$$X_c = X - \mathbf{1} \mu^\top.$$

$$(1) \text{ Mean-matching term: } r = X^\top w - \mu, \quad (1)$$

$$\nabla_w f_1(w) = \nabla_w \|r\|_2^2 = 2 X r = 2 X (X^\top w - \mu). \quad (2)$$

$$(2) \text{ Covariance-matching term: } C(w) = X_c^\top \text{diag}(w) X_c, \quad E(w) = C(w) - \Sigma, \quad (3)$$

$$f_2(w) = \beta \|E(w)\|_F^2 = \beta \text{trace}(E(w)^\top E(w)). \quad (4)$$

For each coordinate i :

$$\frac{\partial f_2}{\partial w_i} = 2\beta \sum_{j,k} E_{jk}(w) \frac{\partial C_{jk}(w)}{\partial w_i}, \quad \frac{\partial C_{jk}}{\partial w_i} = (X_c)_{i,j} (X_c)_{i,k}.$$

Hence

$$[\nabla f_2(w)]_i = 2\beta \sum_{j,k} E_{jk}(w) (X_c)_{i,j} (X_c)_{i,k} = 2\beta [X_c E(w) X_c^\top]_{ii},$$

and in vector form:

$$\nabla f_2(w) = 2\beta \text{diag}(X_c (X_c^\top \text{diag}(w) X_c - \Sigma) X_c^\top).$$

$$(3) \text{ Linear term: } \nabla_w (-\alpha s^\top w) = -\alpha s. \quad (5)$$

Putting these pieces together yields

$$\nabla f_{\text{smooth}}(w) = 2 X (X^\top w - \mu) + 2\beta \text{diag}(X_c (X_c^\top \text{diag}(w) X_c - \Sigma) X_c^\top) - \alpha s.$$

B Convergence and Complexity

We optimize a composite objective of the form

$$F(w) = f(w) + \tau \|w\|_1 + \delta_{\mathcal{C}}(w),$$

where $f(w)$ is smooth with Lipschitz gradient, $\|\cdot\|_1$ is the nonsmooth term, and $\delta_{\mathcal{C}}$ is the indicator of a convex set $\mathcal{C}$ (simplex + per-class constraints).

Convergence rate. FISTA achieves $F(w^{(k)}) - F(w^*) = O(1/k^2)$, so ε -accuracy requires $k = O(1/\sqrt{\varepsilon})$ iterations [5].

Per-iteration cost. Each FISTA iteration includes two main steps:

- **Gradient step:** computing $\nabla f(w)$ involves:
 - two $n \times d$ mat–vec products with X and its centered version $M = X - \mu$, each $O(nd)$, and
 - forming the covariance gradient $C = M^\top \text{diag}(w)M$ (a $d \times d$ matrix) and multiplying back by M , which together cost $O(nd^2)$.

Hence the overall gradient cost is $O(nd^2)$.

- **Projection step:** projecting onto the constraint set $\mathcal{C}$ via Dykstra’s method alternates between:
 - projection onto the simplex using Condat’s algorithm, $O(n)$,
 - per-class sum adjustment via group averaging, $O(n)$.

With projection tolerance ε_p , Dykstra requires $O(\log(1/\varepsilon_p))$ iterations, giving a total projection cost of $O(n \log(1/\varepsilon_p))$.

Thus, each outer iteration costs

$$O(nd^2) + O(n \log(1/\varepsilon_p)) \approx O(nd^2),$$